OBJECT ORIENTED PROGRAMMING CCS20704, DCS20504, CCS23803

Received Date : 3 September 2026
Submission Due Date: 23 October 2026
Lecturer: Dr. Jamal Abdullahi
Weightage : 15%
Semester : SEPTEMBER 2026

Instruction to students:

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

Student declaration:		
I declare that: • ***This assignment is my/our own work.*** • ***I/we understand what is meant by plagiarism.*** • ***My lecturer has the right to deduct my marks in case of:*** - ***Late submission*** - ***Any plagiarism found in my assignment.***		
NAME: SUGANTHAN GOBI **MATRICS NO: 012025070326** 	**NAME:Vegneswaran Kanawadi** **MATRICS NO: 012025021326** 	**NAME:RISHINDRAN GEEVANANTHAN** **MATRICS NO: 012026070861** 

Table of Contents

Section	Page
1. Introduction and project overview	3
2. Object-oriented design and encapsulation	3-4
3. Car class analysis	5
4. Driver class analysis	6
5. Passenger class analysis	7
6. Main class, and interaction flow	8
7. Program Implementation and Output	9-10
8 . Testing, evaluation and conclusion	11-12

1. Introduction and project overview

This project demonstrates object-oriented programming (OOP) through a simple ride-booking scenario. A passenger requests a journey, a licensed driver performs the trip, and a car provides transport. The Java application separates these responsibilities into three classes: **Car**, **Driver**, and **Passenger**. A fourth class, **Main**, creates objects and brings them together in one demonstration.

The design focuses on the course topics of classes, objects, constructors, methods and encapsulation. Rather than placing every action inside one long main method, each class stores its own information and provides operations related to its role. This makes the code easier to follow and helps demonstrate why separating responsibilities is useful.

Project files: Car.java, Driver.java, Passenger.java and Main.java. The uploaded project also includes a README, a presentation guide and a sample-output file. This report discusses the supplied source code rather than a different proposed implementation.

Project objectives

The project aims to create three meaningful objects with seven private attributes each; demonstrate default and parameterised constructors; access and update data through getters and setters; perform useful operations through class methods; and use selection and looping structures to simulate a completed ride.

2. Object-Oriented Design and Encapsulation

A **class** is a blueprint defining attributes and methods. An **object** is an instance created from that class. For example, **Car** describes properties such as brand, model and fuel level, while **car2** is a specific Proton Saga object created in Main.java. The same idea applies to driver2 and passenger2.

Encapsulation means keeping an object's internal data private and controlling how other code accesses it. All seven fields in each of the three project classes use the **private** keyword. The public getters read attributes; the setters update them. Some setters also reject invalid values, such as a negative car speed or a wallet balance below zero.

Class relationship and program flow

Program component	Responsibility	How it connects
Car	Stores vehicle state, starts/stops the engine and drives	Main checks fuel and calls car actions
Driver	Stores personal and licence data, checks driving eligibility and counts trips	Main calls canDrive() and completeTrip()
Passenger	Stores booking details, wallet amount and booking status	Main calls bookRide() and finishRide()
Main	Creates objects, makes decisions and repeats driving actions	Coordinates all three objects

Two kinds of constructors

Each class offers a **default constructor** to create an object with starting values and a **parameterised constructor** to supply values at creation time. Main.java demonstrates both approaches: car1, driver1 and passenger1 start with defaults and are modified with setters; car2, driver2 and passenger2 are created using arguments.

Design boundary

The three objects interact through Main.java rather than storing direct references to one another. This approach is sufficient to illustrate introductory OOP, although a larger transport system might also define a Ride class to manage the full booking lifecycle.

3. Car Class Analysis

The **Car** class models a vehicle and its changing state during a journey. It has seven private attributes and exposes seven matching getters and setters. The constructor provides either default values or values supplied for a particular car.

Attribute	Type	Purpose
brand, model, color	String	Identify and describe the vehicle
year	int	Stores model year
fuelLevel	int	Tracks available fuel units
speed	int	Tracks current speed
engineOn	boolean	Shows whether the engine is running

Methods and meaningful behaviour

startEngine() checks whether the fuel level is above zero before setting engineOn to true. **drive()** checks two conditions: the engine must be on, and fuel must be at least 10 units. When both conditions are met, speed increases by 20 and fuel decreases by 10. **stopEngine()** switches the engine off and resets speed to zero. **showCar()** prints the current vehicle details.

For the parameterised object, the starting fuel level is 50. After three successful calls to drive(), the fuel level becomes 20, and the speed changes from 0 to 20, 40 and 60 km/h. The final stopEngine() call returns speed to zero. These values are visible in the verified console output.

Selected code explanation

`if (engineOn && fuelLevel >= 10)` is a selection condition: driving is permitted only when both conditions are true. The setter `setFuelLevel()` accepts values only from 0 to 100, preventing a caller from using that setter to assign an out-of-range fuel percentage.

Limitation

The car model is intentionally simplified: a successful drive always uses ten fuel units and increases speed by twenty, regardless of journey distance. The parameterised constructor assigns its arguments directly without applying the setters' validation. As a result, invalid initial values are possible even though some later setter calls are checked.

4. Driver Class Analysis

The **Driver** class represents the person responsible for operating the vehicle. It keeps personal information and simple driving-related records. Its seven private attributes can be accessed through seven getters and seven setters.

Attribute	Type	Purpose
name	String	Driver identification
age	int	Age used in eligibility check
licenseNumber	String	Recorded licence identifier
phone, city	String	Contact and location details
yearsExperience	int	Years of experience
tripsCompleted	int	Completed-journey counter

Methods and meaningful behaviour

canDrive() returns true when the driver is at least 18 years old and licenseNumber is not the literal string "None". **completeTrip()** increases the tripsCompleted counter by one and prints a confirmation. **showDriver()** displays the stored details.

In Main.java, driver2 is created as Ali, aged 22, with licence number D200 and zero completed trips. The canDrive() check returns true for these values. After the simulated ride, completeTrip() changes the count from zero to one.

Selected code explanation

`return age >= 18 && !licenseNumber.equals("None");` combines a numeric age condition with a string comparison. This demonstrates returning a boolean value from a method and then using that result in another class.

Limitations

This is an illustrative eligibility check, not an official licence validation. It does not confirm whether a licence is genuine, valid or appropriate for a specific vehicle. The test also assumes licenseNumber is not null; if it is null, the equals call would fail. As with the other classes, the parameterised constructor does not validate the input arguments before storing them.

5. Passenger Class Analysis

The **Passenger** class represents a customer booking a ride. It stores contact details, travel locations, wallet balance and whether a booking is active. All seven attributes are private, and every attribute has a getter and a setter.

Attribute	Type	Purpose
name	String	Passenger name
age	int	Passenger age
phone	String	Contact detail
pickup, destination	String	Journey start and end points
walletBalance	double	Available credit in RM
booked	boolean	Whether a ride has been booked

Methods and meaningful behaviour

bookRide(double fare) checks that no ride is already booked, the fare is positive and the wallet has enough funds. If all conditions pass, it deducts the fare, sets booked to true, prints a message and returns true. Otherwise, it prints a rejection message and returns false. **finishRide()** clears the booking flag when a booking exists, while **showPassenger()** displays current details.

The passenger Aina is initially created with RM30.00, but Main.java updates this balance to RM40.00 using setWalletBalance(). A successful booking with fare RM12.00 leaves RM28.00. Once the trip finishes, booked becomes false again. This demonstrates how a method can change multiple related fields as part of a single action.

Selected code explanation

`if (!booked && fare > 0 && walletBalance >= fare)` links three requirements. The object uses its own internal wallet and booking data, which is more reliable than allowing unrelated code to deduct the fare without checking the booking state.

Limitations

The project does not calculate a real distance-based fare or store payment records. Because money is represented using double, it could also introduce rounding issues in a larger financial application. A production system would require stronger payment handling and a more detailed ride state.

6. Main Class and Interaction Flow

Main.java shows how separate classes work together. It first creates three objects through the default constructors and changes selected fields through setters. Their details are printed to show the starting state. It then creates a second set using parameterised constructors and updates the car colour and passenger wallet.

Journey sequence

Step	Action	Relevant OOP or control feature
1	Create car2 (Proton Saga), driver2 (Ali) and passenger2 (Aina)	Constructors and objects
2	Change car colour to Black and wallet to RM40	Setters; read information with getters
3	Check driver2.canDrive(), car fuel ≥ 30 and passenger2.bookRide(12.0)	if statement; short-circuit AND
4	Start the car engine	Method call on Car object
5	Run car2.drive() three times	for loop; changing object attributes
6	Stop the car, complete the driver trip and finish passenger ride	Methods across three objects
7	Print all updated object details	Getters inside display methods

Why the control structures matter

The **if** statement controls whether the ride proceeds. If any condition fails, the program reports that the ride cannot continue. The **for** loop repeats the driving operation exactly three times, reducing repeated code and making it easy to observe how speed and fuel change. This implementation uses **if** and **for**; it does not contain a switch statement or a while loop, which are examples rather than mandatory choices in the brief.

Object interaction

The driver is checked before the car begins moving, the passenger pays at booking, and the completed journey updates both the driver's trip count and the passenger's booking status. This links the behaviour of three distinct classes in one coherent demonstration.

The program is a fixed console demonstration rather than an interactive menu: sample objects and fares are defined within Main.java. This is suitable for showing the required OOP concepts but is less flexible than accepting input from a real user.

7. Program Implementation and Output

```java
// Main class: shows how Car, Driver and Passenger work together
public class Main {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        // Create all three objects using default constructors
        Car car1 = new Car();
        Driver driver1 = new Driver();
        Passenger passenger1 = new Passenger();

        // Use setters to update the default objects
        car1.setBrand(brand: "Perodua");
        car1.setModel(model: "Myvi");
        driver1.setName(name: "Sara");
        driver1.setLicenseNumber(licenseNumber: "D100");
        passenger1.setName(name: "Meena");
        passenger1.setPickup(pickup: "College");
        passenger1.setDestination(destination: "Library");

        System.out.println("=== DEFAULT OBJECTS ===");
        car1.showCar();
        driver1.showDriver();
        passenger1.showPassenger();
```

Figure 1: Object Creation and Initialisation in Main.java

The code demonstrates the creation of Car, Driver and Passenger objects using default constructors, followed by the modification of their attributes using setter methods.

```
=== DEFAULT OBJECTS ===
Car: Perodua Myvi
Color: White, Year: 2020
Fuel: 30, Speed: 0
Engine on: false
Driver: Sara, Age: 18
License: D100, Phone: Not set
City: Unknown, Experience: 0 years
Trips completed: 0
Passenger: Meena, Age: 18
Phone: Not set
From: College To: Library
Wallet: RM0.0, Ride booked: false
```

Figure 2: Initial Program Output

The output displays the initial information of the Car, Driver and Passenger objects before the ride-booking process begins.

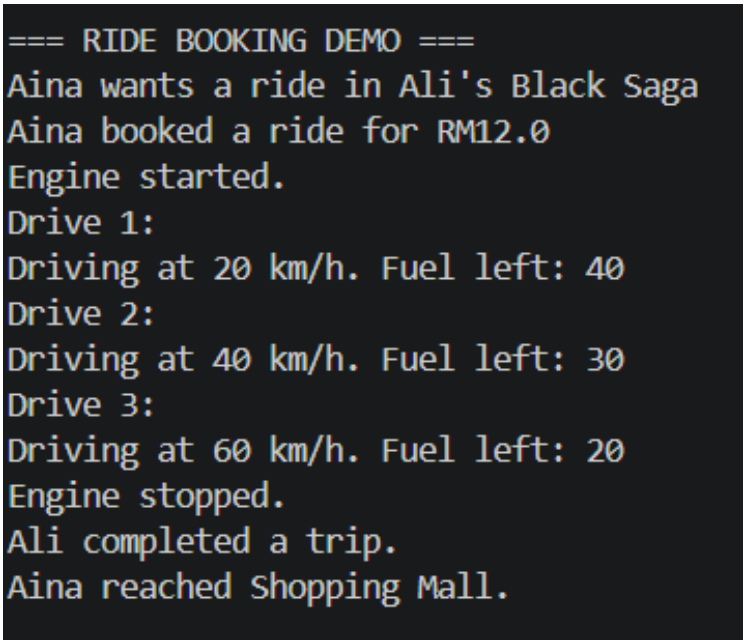


Figure 3: Ride Booking and Driving Process
The output demonstrates the ride-booking process, including fare calculation, engine operation, driving steps and successful trip completion.

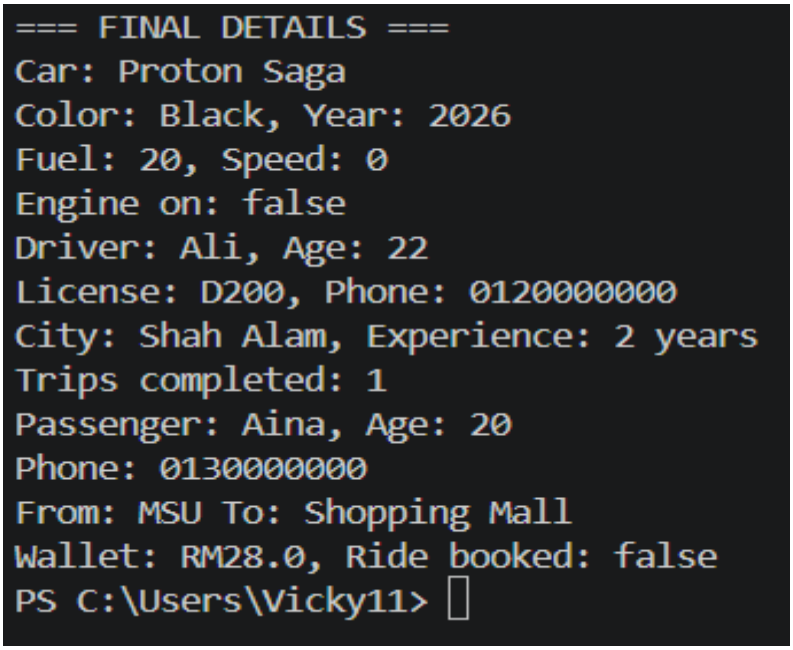


Figure 4: Final Object Information After Ride Completion The output shows the updated Car, Driver and Passenger information after the completed ride, including the remaining fuel, total completed trips and passenger's wallet balance.

8 . Testing, Evaluation and Conclusion

Compilation and actual run

The supplied Java files were compiled together using javac Car.java Driver.java Passenger.java Main.java and executed using java Main. This compilation and one complete run succeeded in the preparation of this report. The table below records key observations from the actual console output, not hypothetical results.

Check	Observed outcome
Initial state	Default objects print Perodua Myvi, driver Sara and passenger Meena.
Booking	Aina booked a ride for RM12.0.
Driving loop	Three drive messages: speed 20, 40 and 60; remaining fuel 40, 30 and 20.
Trip completion	Engine stopped; Ali completed a trip; Aina reached Shopping Mall.
Final state	Car speed 0, fuel 20, driver trips 1, passenger wallet RM28.0, booked false.

Rubric requirement review

Each of Car, Driver and Passenger defines seven private attributes, a default constructor, a parameterised constructor, seven getters and seven setters. Car contains four additional action/display methods; Driver and Passenger each contain three. Main uses an if condition and a for loop, calls methods on all three objects, and demonstrates reading and updating attributes through getters and setters.

Critical evaluation

The current implementation demonstrates the assignment’s OOP fundamentals clearly, especially encapsulation and the connection between object state and behaviour. However, the constructor inputs are not validated, driver licence validation is basic, and the simulation does not model fare calculation, real travel distance or persistent booking records. Only one complete example run was verified for this report; separate boundary-case tests have not been claimed as performed.

Conclusion

This Java project shows how a ride-booking situation can be divided into manageable objects. Car handles vehicle behaviour, Driver checks simple eligibility and records trips, and Passenger manages the booking and wallet. Main coordinates these operations using constructors, getters, setters, selection and repetition. The result is a working introductory OOP demonstration with opportunities for stronger validation and more realistic ride management.

Submission note: Add the actual member names, matric numbers and photos to the original assignment cover sheet. The source-code ZIP must also be submitted separately through eKlas. The attached code identifies the classes as Member 1 (Car), Member 2 (Driver) and Member 3 (Passenger), but does not identify the students by name.

GITHUB

https://github.com/suganthangobi-max/OOP-Assignment-1.git

RUBRIC for Assignment One

CLO1 – Describe the concepts of object oriented programming. (C5, PLO1)

COMPONENT	CRITERIA / WEIGHTAGE (%)	LEVELS of ACHIEVEMENTS/ DESCRIPTORS					
		Excellent (4)	Good (3)	Average (2)	Needs Improvement (1)	Not Attempted (0)	Marks
CONTENT (20%)	Object Design Understanding (5%)	Clear and accurate explanation of class, attributes, methods, and constructors using correct OOP terms	Mostly correct explanation with minor gaps	Basic explanation but lacks clarity	Limited or incorrect explanation	No explanation	
	Encapsulation Concept (5%)	Strong understanding; correct use of private attributes and getters/setters	Good understanding with minor issues	Basic understanding but inconsistent	Weak understanding	No attempt	
	OOP Principles Explanation (5%)	Clearly explains class, object, and methods with examples	Mostly clear explanation	Basic explanation	Poor explanation	Not explained	
	Presentation Knowledge & Understanding (5%)	Confident explanation using OOP terms; answers questions clearly	Good understanding	Basic explanation	Weak understanding	No participation	
PROCESS (40%)	Class Implementation (10%)	Fully meets requirements (≥7 attributes, 2 constructors, getters/setters, ≥5 methods)	Mostly complete with minor issues	Partially complete	Incomplete	Not attempted	
	Encapsulation Implementation (10%)	Majority attributes private; proper getters/setters used	Mostly correct	Some inconsistencies	Poor implementation	Not implemented	

	Control Structures in Main Class (10%)	Effective use of if/switch and loops integrated with object	Correct with minor issues	Basic use	Minimal/incorrect use	Not used	
	Code Logic & Interaction (10%) + (Presentation)	Clear logic; meaningful interaction using methods	Mostly logical	Basic logic	Weak logic	Not functional	
PRODUCT (40%)	Functionality & Output (5%)	Program runs correctly with accurate output	Mostly correct output	Basic output with issues	Incorrect output	Not working	
	Creativity & Object Design (5%)	Highly creative, real-world, advanced methods	Creative and relevant	Basic design	Generic design	Not appropriate	
	Code Readability & Comments (5%)	Clean, well-organized, fully commented code	Mostly readable	Limited comments	Poor readability	No comments	
	Report Quality & Documentation (10%)	Well-structured report; clear explanation; proper formatting	Mostly organized	Acceptable	Poor documentation	Not submitted	
	Presentation & product Knowledge (15%)	Confident explanation using OOP terms; answers questions clearly	Good understanding	Basic explanation	Weak understanding	No participation	
						TOTAL	